

Universidade Federal de Sergipe (UFS)
Departamento de Computação

Disciplina: Engenharia de Software II (2025.2)
Docente: Prof. Dr. Glauco de Figueiredo Carneiro

Atividade 3 - Etapas 2 e 3
Implementação e Análise Crítica de CI/CD
Projeto Objeto: *Scrapegraph-ai*

Equipe e Temas de Apresentação:

Integrante(s)	Matrícula	Responsabilidade na Apresentação
Rafael Gomes O. Santos / Breno Silva do Nascimento	202300095730 / 202300038968	Arquitetura do Workflow Automatizado.
José Gabriel R. G. de Almeida	202300095599	O Fluxo de Dados: Uma Linha de Montagem Virtual.
Pedro Joaquim S. Silveira / Mateus da Silva Barreto	202300038897 / 202300038879	Decisão Estratégica: Por que escolhemos a "Estratégia C"?
José Victor Ribeiro de Jesus / Maria Eduarda M. da Silva	202300038799 / 202300038860	2*O Impacto Humano e Técnico da Automação.
Cauan Teixeira Machado	202300038627	Introdução e Coordenação do Repositório Fork.

Data de Entrega: 06 de Fevereiro de 2026

1 Dados da Implementação

- **Projeto Original:** Scrapegraph-ai
- **URL do Repositório (Fork):** https://github.com/mateus-barreto-dev/Engenharia_de_Software-II_Etapa-3
- **Link do Green Build (Actions):** https://github.com/mateus-barreto-dev/Engenharia_de_Software-II_Etapa-3/actions/runs/21724918250

2 Tutorial de Execução e Implementação

Para a realização das etapas de implementação, a equipe seguiu o roteiro técnico abaixo:

1. **Criação do Fork:** O repositório foi replicado para o ambiente da equipe para permitir a inserção de novos fluxos.
2. **Configuração do Workflow:** Criamos o arquivo `.github/workflows/quality_gate.yml` definindo os gatilhos para *push* e *pull request*.
3. **Ambiente de Execução:** O sistema aloca uma máquina virtual *Ubuntu Latest* com Python 3.10.
4. **Execução do Smoke Test:** O script `ci_demo.py` valida a integridade do ambiente e das bibliotecas essenciais.

3 Arquitetura do Workflow

```
name: Quality Gate - ScrapeGraphAI
on: [push, pull_request]
jobs:
  demo-validation:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v4
        with:
          python-version: '3.10'
      - run: python ci_demo.py
```

4 Discussão Crítica e Evolução de Software

A decisão pela **Estratégia C** foi baseada em *trade-offs* técnicos, visto que o projeto original exige chaves de API pagas e alto poder computacional (LLMs) que excedem as cotas gratuitas do GitHub.

Nossa automação combate o *Software Decay* ao agir como um sistema imunológico que detecta anomalias antes que elas degradem o sistema. Além disso, garante segurança psicológica no *onboarding*, permitindo que novos desenvolvedores recebam feedback imediato e autônomo sobre a qualidade de suas contribuições.

5 Diagrama do Pipeline Implementado

O fluxo abaixo representa visualmente as etapas automatizadas na nossa esteira de Integração Contínua (CI). O diagrama detalha o processo desde o gatilho inicial até a obtenção do sinal verde, garantindo que o ambiente e o "esqueleto" funcional do *ScrapeGraphAI* estejam íntegros.

Diagrama do Pipeline



Figura 1: Fluxo de Automação (Estratégia C): Gatilho, Preparação e Smoke Test.

Como ilustrado na Figura 1, o pipeline executa quatro etapas críticas:

1. **Gatilho (Trigger):** Ativado por *Push* ou *Pull Request*.
2. **Checkout:** O código é baixado no *runner* (máquina virtual).
3. **Setup Python:** O ambiente isolado é preparado com a versão 3.10.
4. **Smoke Test:** O script `ci_demo.py` é executado para validar a carga das bibliotecas e a lógica base.